1

# “Gang of Four” (GoF) Patterns

The Fundamental OO Design Patterns

Includes content adapted from *Design Patterns: Elements of Reusable Object-Oriented Software*, 1994, Addison-Wesley && *Java Design Patterns, 2022, Apress*



## Design Patterns

- First described in 1994 by a group of four well-known software engineering researchers:
  - Erich Gamma, John Vlissides, Ralph Johnson, Richard Helm
  - This group came to be known as the “Gang of Four”
- The first time the concept of “patterns” was used to describe and codify software development knowledge.



## Design Patterns

- First described in 1994 by a group of four well-known software engineering researchers:
  - Erich Gamma, John Vlissides, Ralph Johnson, Richard Helm
  - This group came to be known as the “Gang of Four”
- The first time the concept of “patterns” was used to describe and codify software development knowledge.



# Design Patterns

- The 23 GoF patterns are classified into three different groups according to purpose:
  - Creational patterns
  - Structural patterns
  - Behavioral patterns

| | | | Purpose | |
|---|---|---|---|---|
| | | **Creational** | **Structural** | **Behavioral** |
| **Scope** | **Class** | 1. Factory Method | 1.1. Adapter(class) | 1. Interpreter<br>2. Template Method |
| | **Object** | 2. Singleton<br>3. Prototype<br>4. Builder<br>5. Abstract Factory | 1.2. Adapter(object)<br>2. Proxy<br>3. Flyweight<br>4. Composite<br>5. Bridge<br>6. Facade<br>7. Decorator | 3. Observer<br>4. Strategy<br>5. Command<br>6. Iterator<br>7. Memento<br>8. State<br>9. Mediator<br>10. Visitor<br>11. Chain of Responsibility |

# Design Patterns

- They are also classified according to scope (class or object)
  - Patterns with “class” scope are created using static, compile-time mechanisms such as inheritance.
  - Patterns with ”object” scope rely on dynamic and runtime mechanisms

| | | | Purpose | |
|---|---|---|---|---|
| | | **Creational** | **Structural** | **Behavioral** |
| **Scope** | **Class** | 1. Factory Method | 1.1. Adapter(class) | 1. Interpreter<br>2. Template Method |
| | **Object** | 2. Singleton<br>3. Prototype<br>4. Builder<br>5. Abstract Factory | 1.2. Adapter(object)<br>2. Proxy<br>3. Flyweight<br>4. Composite<br>5. Bridge<br>6. Facade<br>7. Decorator | 3. Observer<br>4. Strategy<br>5. Command<br>6. Iterator<br>7. Memento<br>8. State<br>9. Mediator<br>10. Visitor<br>11. Chain of Responsibility |



## Structural Patterns

- Focus on creating larger structures by grouping several smaller classes and objects, usually by means of composition or inheritance.
- These patterns improve the flexibility of efficiency of the program.



## Behavioral Patterns

- These concern handling algorithms and assigning responsibilities to classes.
- These patterns describe several ways in which objects are connected and collaborate to achieve a certain goal.

1

# Creational Patterns

How to create the objects in your program



## Creational Patterns

- These abstract the instantiation process.
- Main concern is assigning the object creation responsibility.
  - “Where should I use the *new* keyword?”
- These can typically improve coupling by separating object creation logic from the client code that uses those objects.



## Creational Patterns

1. **Factory method**
2. Abstract Factory
3. Prototype
4. Builder
5. Singleton

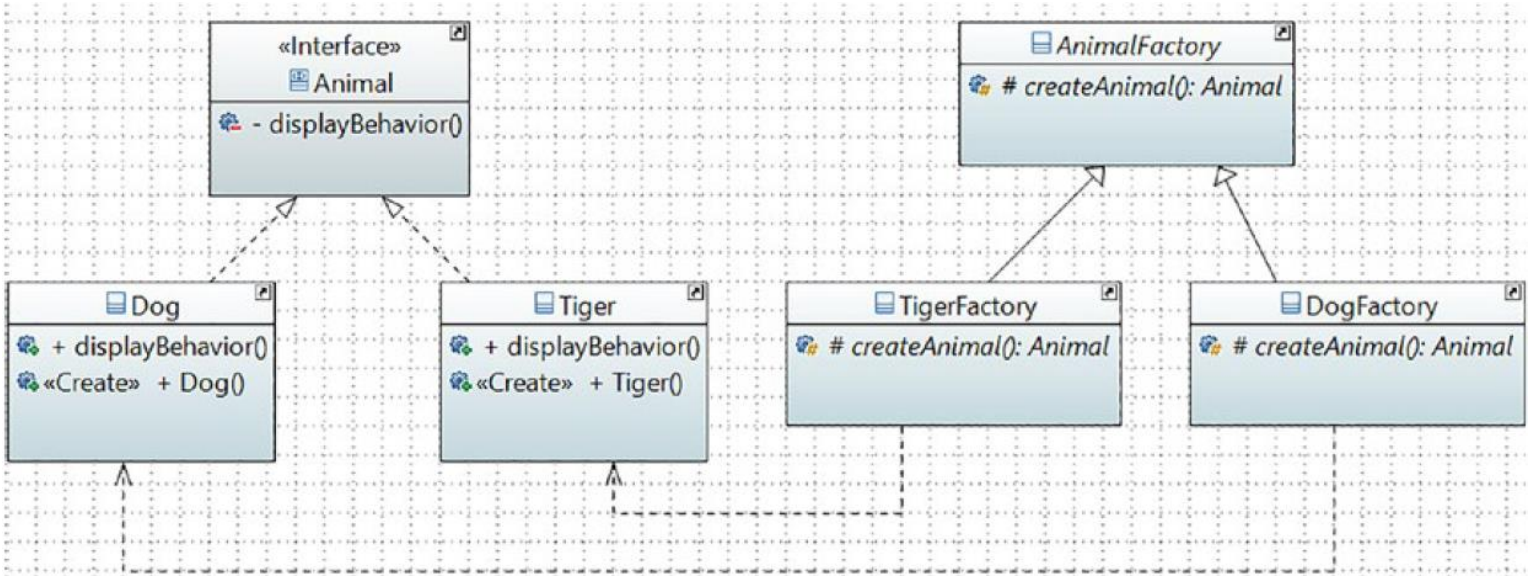


## Factory Method

- Create an object without exposing instantiation logic to client code.
- It defines an interface for creating an object, but lets subclasses decide which class to instantiate.
  - The abstract base class or interface is also known as the *Creator*.

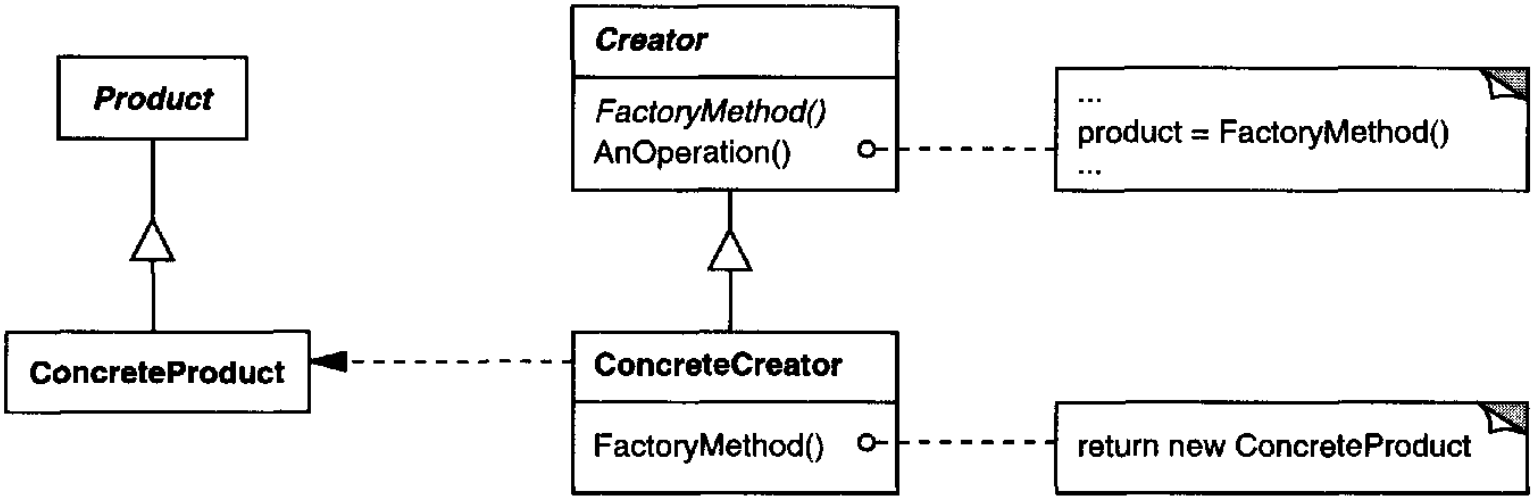
**Java API example:** The `iterator()` method of the `AbstractCollection<E>` (or `Collection<E>`) is an example of a factory method in Java.

# Factory Method



# Factory Method

- Generic Structure





# Factory Method

- **Advantages:**
  - Separates code that varies from code that does not vary.
  - A client can use a factory method without knowing how the actual type of object is created, reducing coupling.
  - Because the code is not tightly coupled, you can add new subclasses of the appropriate type at any time in the system without modifying the existing architecture (the Open/Closed Principle).



## Creational Patterns

1. Factory method
2. **Abstract Factory**
3. Prototype
4. Builder
5. Singleton

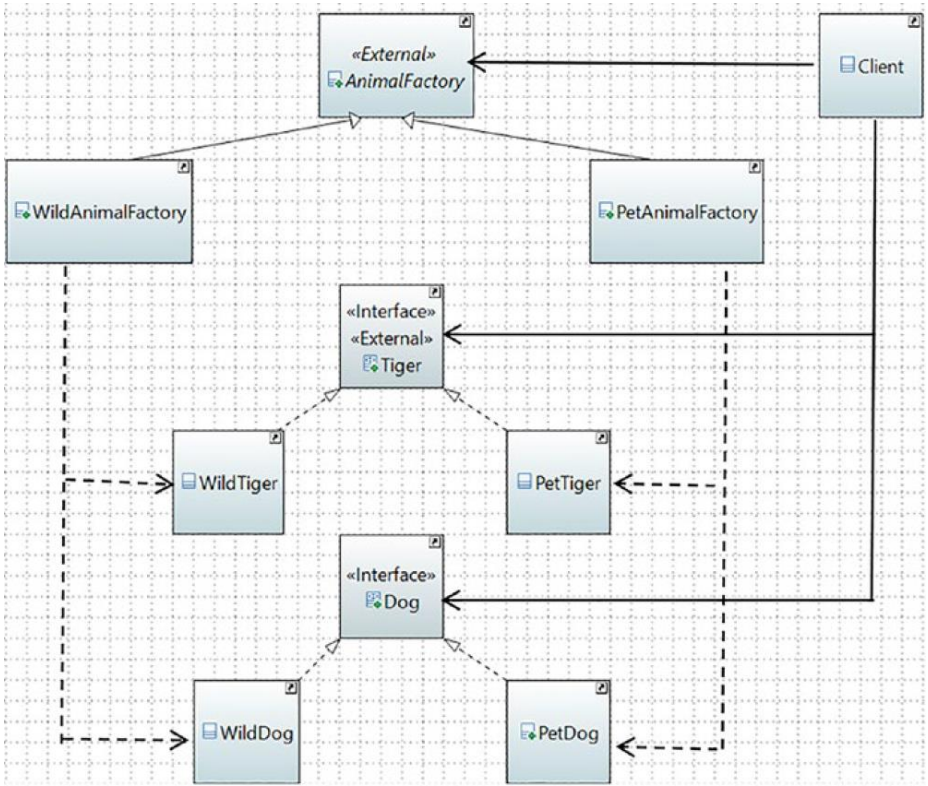


## Abstract Factory

- It provides an interface for creating families of related or dependent objects without specifying their concrete classes. It is a *factory of factories:*
  - Encapsulate a group of individual factories that have a common theme
  - This pattern helps you interchange specific implementations without changing the code that uses them, even at runtime.

**Java API example:** The `newInstance()` method of `javax.xml.parsers`.

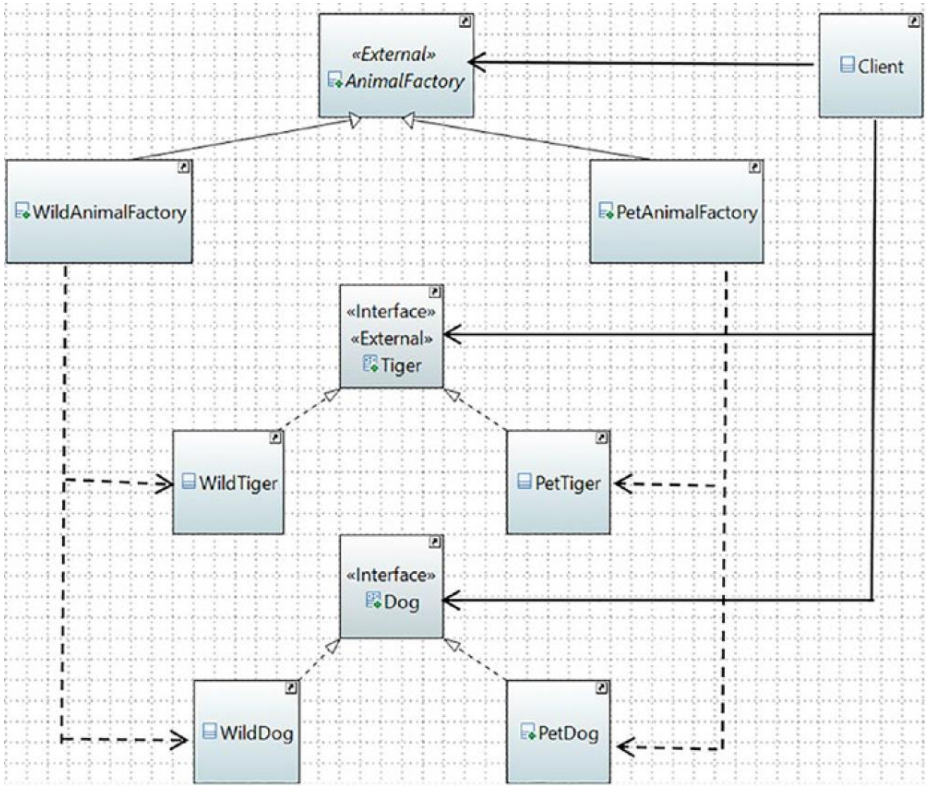
# Abstract Factory




Client Code

```
AnimalFactory animalFactory;
animalFactory = new WildAnimalFactory();
Dog dog =
animalFactory.createDog("white");
Tiger tiger =
animalFactory.createTiger("golden");
dog.displayMe();
tiger.aboutMe();
```

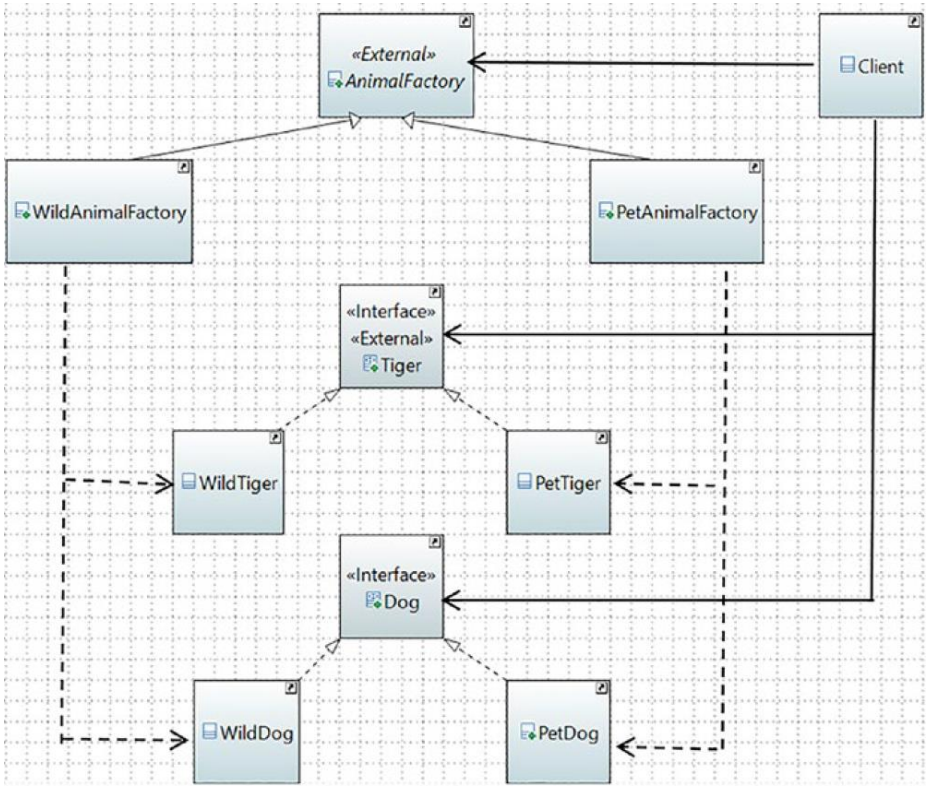
# Abstract Factory




Client Code

```
AnimalFactory animalFactory;
animalFactory = new PetAnimalFactory();
Dog dog =
animalFactory.createDog("white");
Tiger tiger =
animalFactory.createTiger("golden");
dog.displayMe();
tiger.aboutMe();
```

# Abstract Factory




Client Code (Using a Factory Provider)

```
AnimalFactory animalFactory;
animalFactory =
FactoryProvider.getFactory("wild");
Dog dog =
animalFactory.createDog("white");
Tiger tiger =
animalFactory.createTiger("golden");
dog.displayMe();
tiger.aboutMe();
```



## Creational Patterns

1. Factory method
2. Abstract Factory
3. **Prototype**
4. Builder
5. Singleton

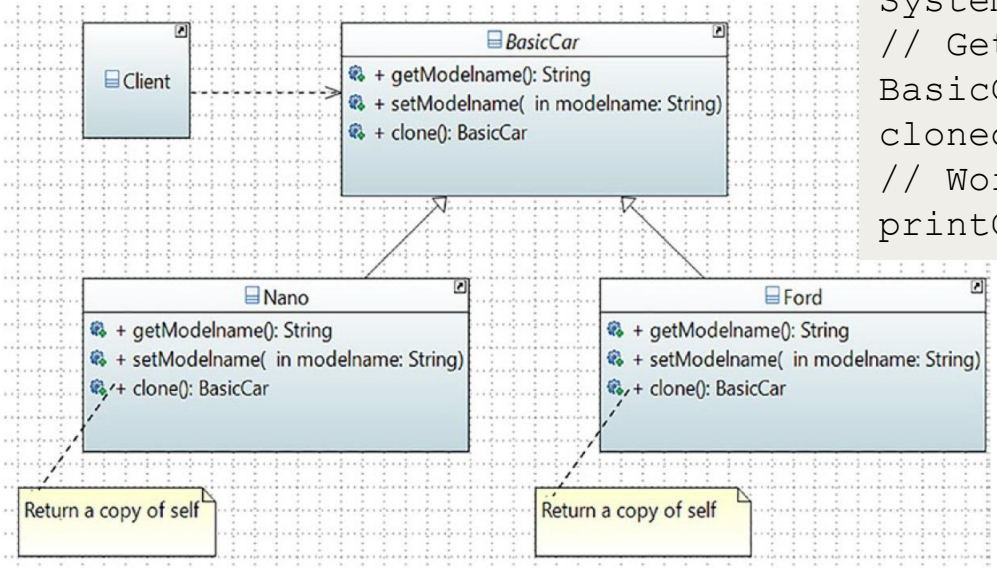


## Prototype

- It specifies the kinds of objects to create using a prototypical instance and creates new objects by copying this prototype.
- Allows reuse of existing objects:
  - Reuses resources
  - Avoid rebuilding an object from scratch.

**Java API example:** The `clone()` method of `java.Object`.

# Prototype




Client Code

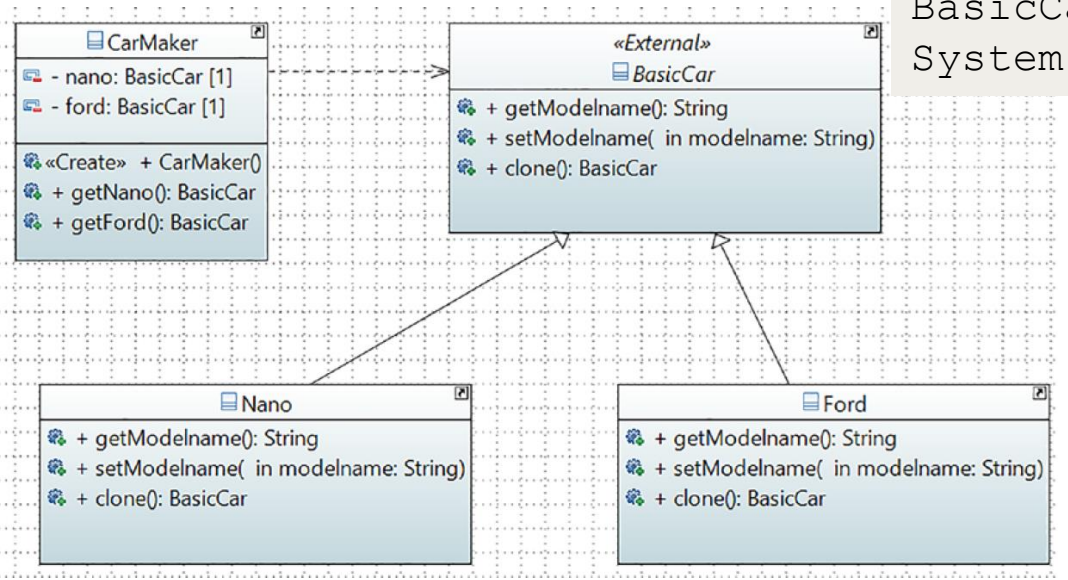
```
BasicCar nano = new Nano("Nano XM624");
System.out.println(nano);
System.out.println("-------");
// Getting a cloned version of Nano
BasicCar clonedCar;
clonedCar = nano.clone();
// Working with the cloned Nano
printCarDetail(clonedCar);
```



# Prototype

Client Code (using a *Maker* object)

```
CarMaker carMaker = new CarMaker();
// Working with a Nano car
BasicCar nano = carMaker.getNano();
System.out.println(nano);
```

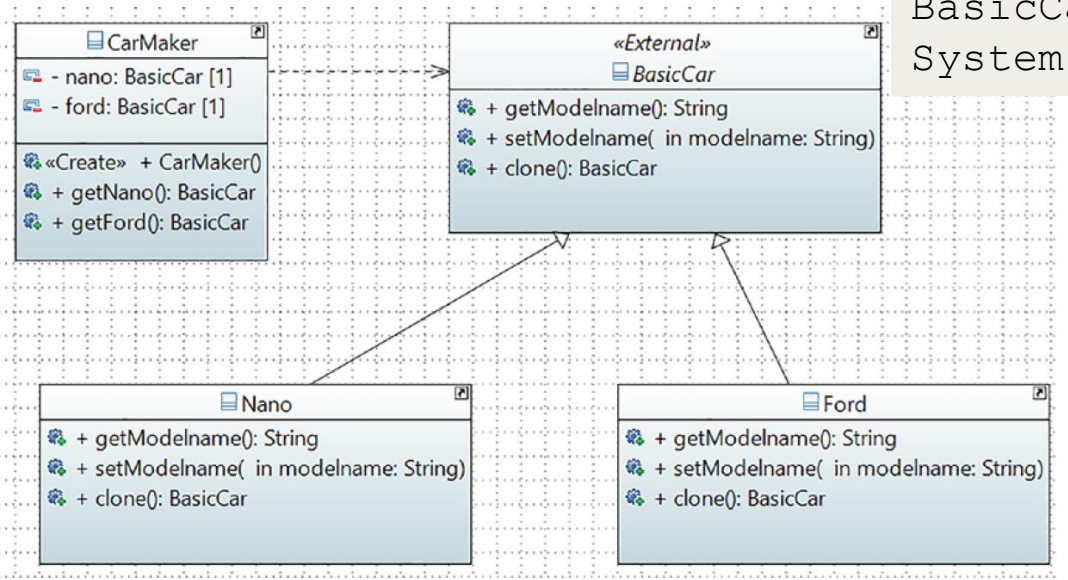




# Prototype

Client Code (using a *Maker* object)

```
CarMaker carMaker = new CarMaker();
// Working with a Nano car
BasicCar nano = carMaker.getNano();
System.out.println(nano);
```

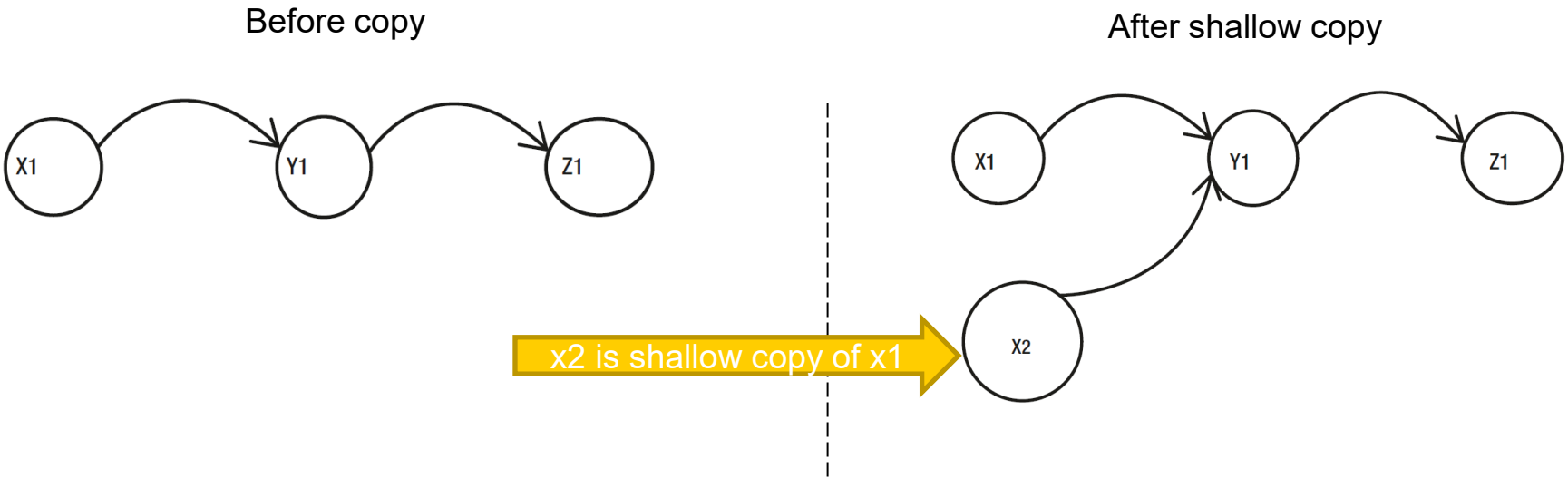



*Other implementation choices, not shown here, include copy-constructors or the use of serialization mechanisms.*



# Prototype

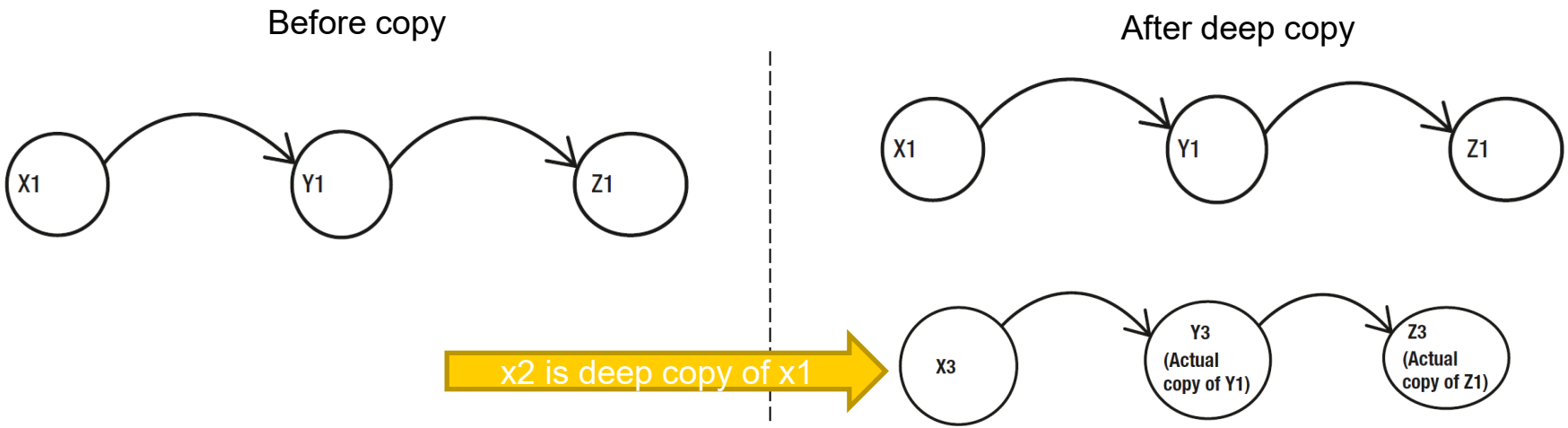
- **Shallow** vs. Deep copy





# Prototype

- Shallow vs. **Deep** copy





# Prototype

- **Shallow vs. Deep copy**
  - A shallow copy is faster and less expensive. It is suitable if your target object has the primitive fields only.
  - A deep copy is expensive and slow. But it is useful if your target object contains fields that have references to other objects.



## Creational Patterns

1. Factory method
2. Abstract Factory
3. Prototype
4. **Builder**
5. Singleton



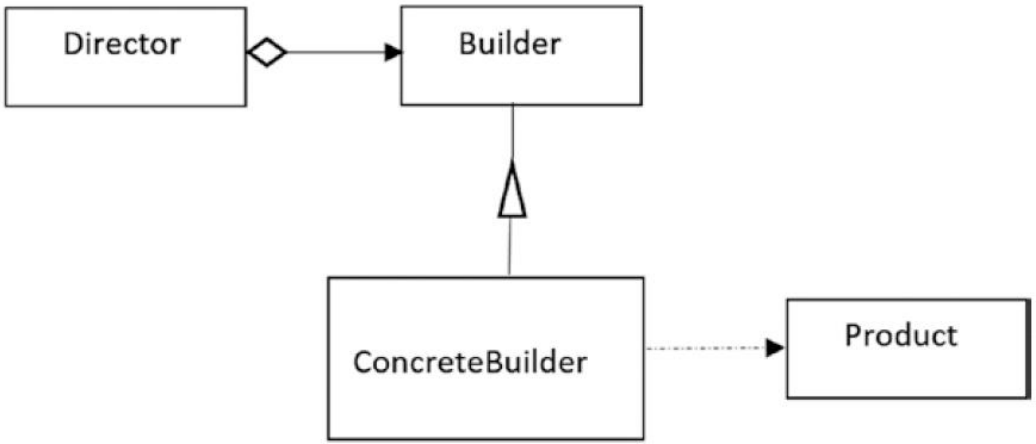
# Builder

- It separates the construction of a complex object from its representation so that the same construction processes can create different representations.
  - Useful for creating complex objects that have multiple parts.
  - The object creation process should be independent of these parts.

**Java API example:** `Java.util.Calendar.Builder`



# Builder

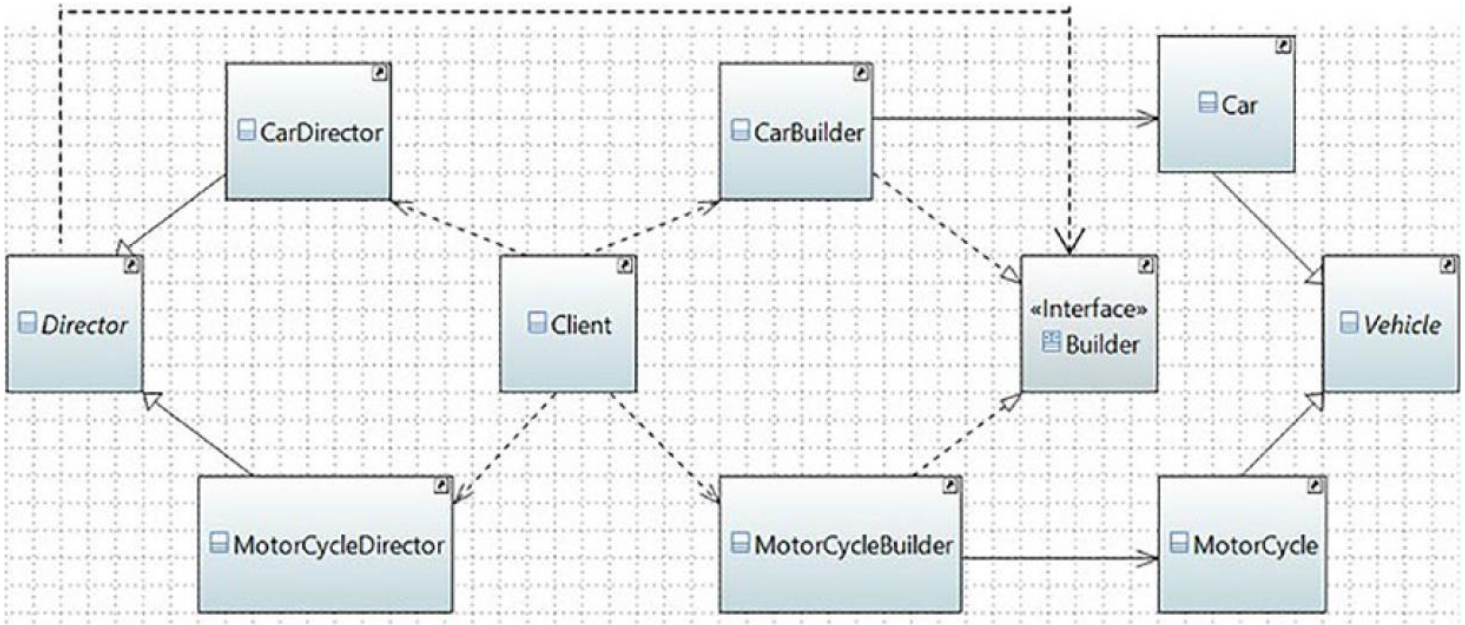



- **Director** – Responsible for specifying the object creation data
- **Builder** – Abstract builder interface
- **ConcreteBuilder** – Responsible for receiving object creation data and creating the requested object on demand.
- **Product** – The object created using the information provided by the director.

# Builder

Client Code

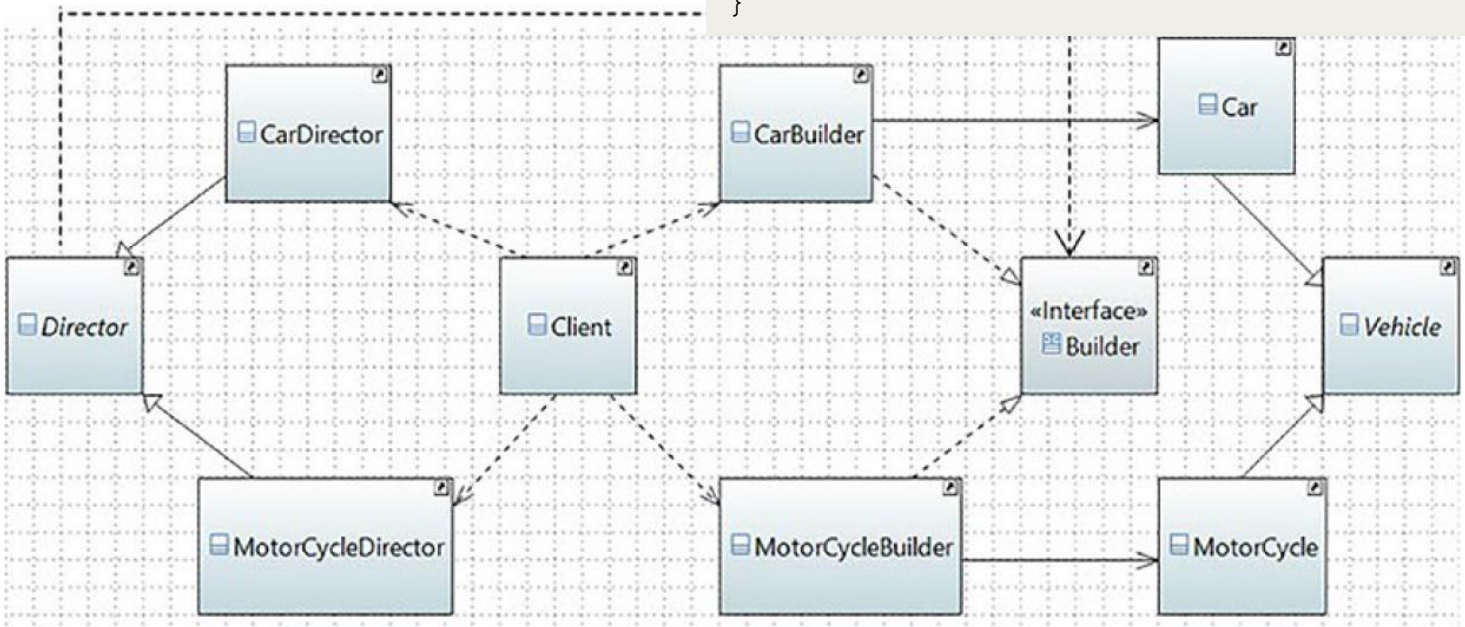
```
Builder builder = new CarBuilder();
Director director = new CarDirector();
Vehicle vehicle=director.instruct(builder);
vehicle.showProduct();
```



# Builder

Director Code

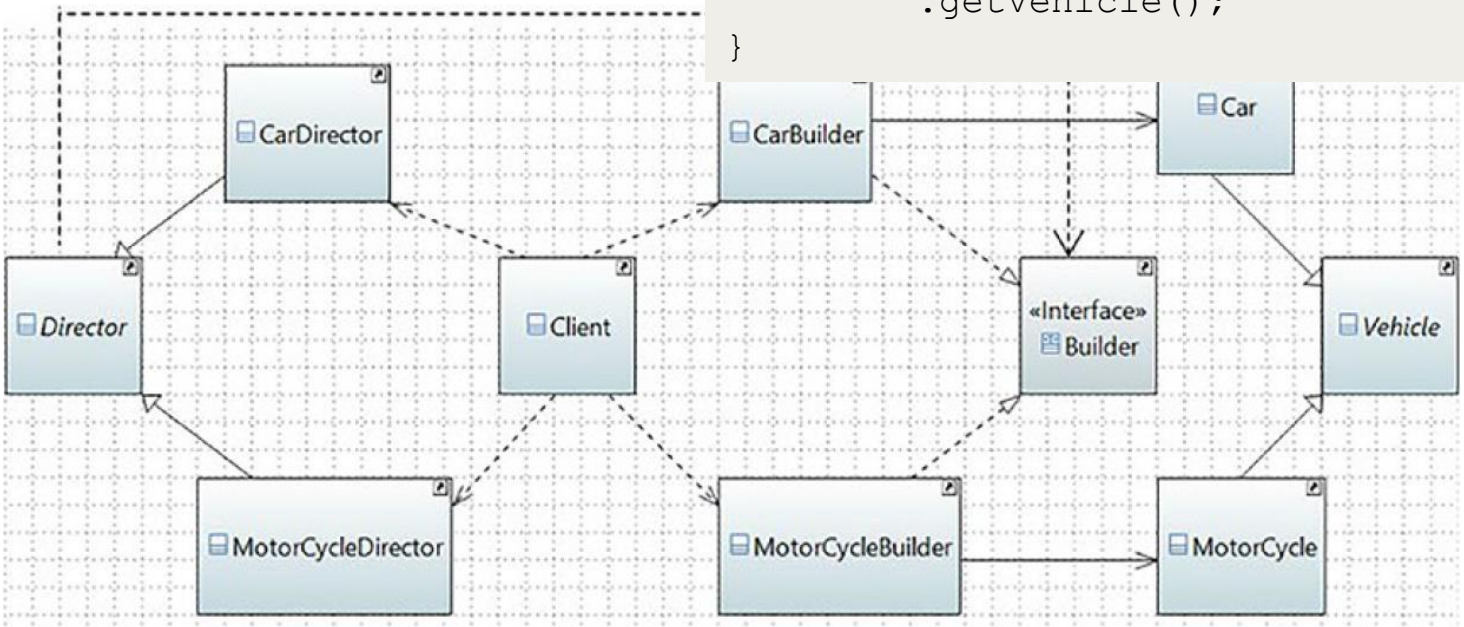
```
public Vehicle instruct(Builder builder) {
builder.buildBody();
builder.insertWheels();
builder.addBrandName();
return builder.getVehicle();
}
```



# Builder

Director Code (*fluent-style Interface*)

```
public Vehicle instruct(Builder builder) {
return builder
         .buildBody()
         .insertWheels()
         .addBrandName()
         .getVehicle();
}
```





# Builder

- **Advantages:**
  - Objects can be built step-by-step.
  - The same construction process (i.e. the same Director) can build different objects (by using different concrete builders)



## Creational Patterns

1. Factory method
2. Abstract Factory
3. Prototype
4. Builder
5. **Singleton**

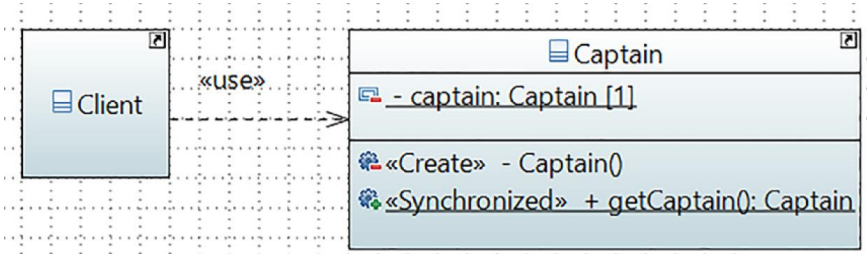


# Singleton

- It ensures that a class has only one instance and provides a global point of access to it.
  - Useful for creating objects that manage or require unique resources.
  - Useful for avoiding unnecessary creation of duplicated data objects.

**Java API example:** `java.lang.Runtime`

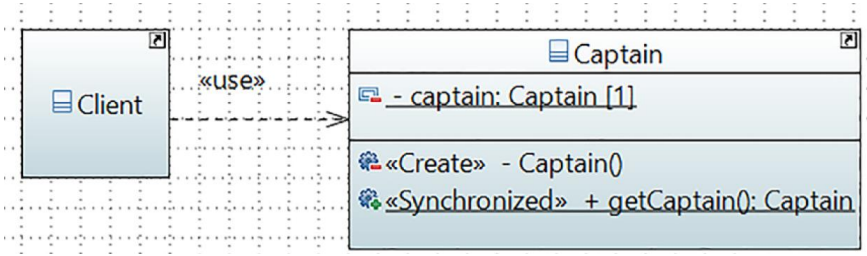
# Singleton




Client Code

```
Captain captain1 = Captain.getCaptain();
System.out.println("Trying to make another
captain for your team:");
Captain captain2 = Captain.getCaptain();
if (captain1 == captain2){
System.out.println("Both captain1 and
captain2 are the same.");
```

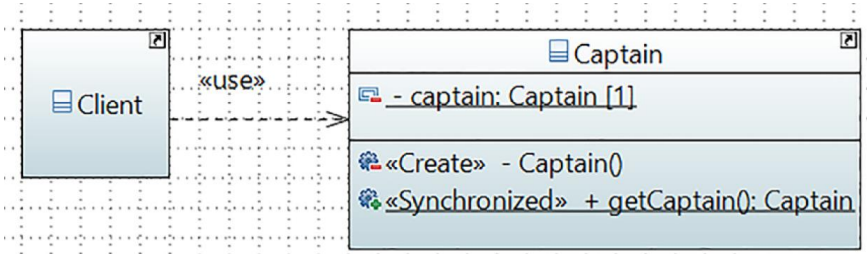
# Singleton




Singleton Code

```
public static synchronized Captain getCaptain() {
// Lazy initialization
if (captain == null)
        captain = new Captain();
return captain;
}
```

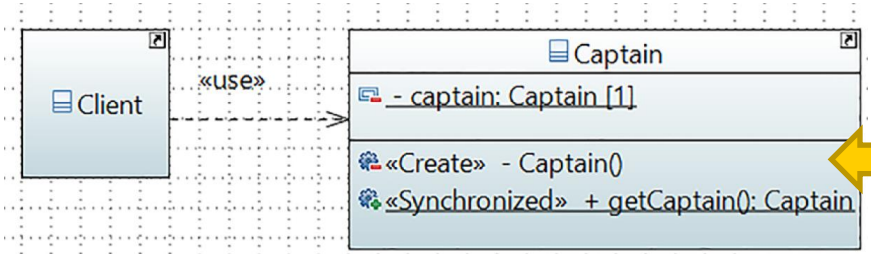
# Singleton




Singleton Code (Eager Initialization)

```
// Early initialization
private static final Captain CAPTAIN_INSTANCE = new Captain();
public static Captain getCaptain(){
return CAPTAIN_INSTANCE;
}
```

# Singleton




Private constructor

Client Code

```
new Captain(); //ERROR
```